

Format and Grading

- **Release:** Tuesday, May 5, 2026, 12:00 PM.
- **Written report due:** 11:59 PM on Sunday, May 10, 2026 (Canvas upload).
- **Oral defense:** 8-minute one-on-one meeting with the instructor on **Monday, May 11, 2026, 2:05–4:05 PM**. All defenses happen in this single window; sign up for a slot in Canvas.

Structure. Four computational problems (P1–P4), two theory problems (P5–P6), and an oral defense.

- **Pick two of the four computational problems P1–P4.** Each required problem is worth **20 pts**. *If you submit work for all four, your best two count toward the required 40 pts and the next-best counts as the bonus.*
- **Optional bonus:** a third computational problem for **+10 bonus pts**. Pick whichever third one you find most interesting; there is no penalty for not attempting a third.
- **Both theory problems P5 and P6 are required.** Each is worth **20 pts**.
- **Oral defense: 20 pts** (rubric at the end of this document).

Required total: 100 pts (40 from your two computational problems + 40 from theory + 20 from oral).

Maximum with bonus: 110 pts. Scores are scaled to a percentage based on the required total — the bonus is genuinely additive.

Directions

- **Exam scope:** Everything from the semester. You may freely reference the lecture slides, course notebooks, and the textbooks (Nocedal & Wright, Boyd & Vandenberghe, Biegler).
- **Problems:** Four large computational problems (P1–P4), then two short theory problems (P5–P6). Each computational problem asks you to *think about implementation, justify an algorithmic choice, and report results*.
- **Data.** A `data/` folder is posted on Canvas with three `.npz` files: `portfolio30.npz` (P1), `wdbc.npz` (P2), `case9.npz` (P3). P4 generates its own random data (no file needed). Each `.npz` file is loadable with `numpy.load`; the relevant snippet appears in the corresponding problem statement. All files are reproducible from `data/generate_data.py` (also posted) which runs in ≤ 10 seconds.
- **Deliverables & submission.**
 - Single PDF with your complete write-up, typed in $\text{\LaTeX}$, with all plots and tables embedded.
 - A `.zip` of code (Python scripts or Jupyter notebooks) containing everything you ran. Code must reproduce every number and every figure in your report.
 - Both uploaded to Canvas by the deadline.
- **What the report should contain, for each computational problem:**
 1. A *short problem formulation* in your own words (1 paragraph).
 2. An *algorithm-selection justification* (1–2 paragraphs): which method did you use, why, what was the alternative you rejected, and why.
 3. *Results*: tables, plots, runtime, iteration counts.
 4. A *discussion* (1 paragraph): what surprised you, what would you do differently next time, how does this connect to the theory.

- **AI policy.** Read carefully.
 - You **may** use AI tools (ChatGPT, Claude, Copilot, etc.) for two specific kinds of work:
 1. *Writing and debugging code* for the computational problems (P1–P4).
 2. *Writing and formatting $LaTeX$* (typesetting equations, fixing compilation errors, formatting tables and figures).
 - You **may not** use AI to do mathematical derivations, proofs, or written discussion. This includes:
 - * The KKT derivations and proofs in Problems 5 and 6.
 - * The gradient/Hessian derivations and algorithm-selection essays inside Problems 1–4.
 - * Any analytical work that demonstrates *your* understanding of the material.
 - **Disclosure (mandatory).** In your written report, include a short subsection titled “AI Use” that states:
 1. Which AI tools you used (e.g., “Claude Sonnet 4.6,” “ChatGPT-4o,” “GitHub Copilot”).
 2. Which problems / subparts you used them on.
 3. One or two representative prompts you submitted.

Failure to disclose AI use when AI was used is an honor-code violation. Disclosing AI use that is within the policy above is *not* penalized.
- **Collaboration:** None. This is an individual exam. Questions about the statement of a problem (not its solution) are welcome during office hours on May 6–8.
- **What I’m looking for:** You are a soon-to-be graduate in the applied optimization world. When a boss hands you a real problem, you need to choose an algorithm, implement it, and tell a story about why you chose it and what the results mean. This exam simulates that: I don’t just want the right answer, I want the *reasoning around* the right answer.
- **Oral defense preparation.** Bring your code and report. I will ask:
 - “Walk me through how you approached Problem [X].”
 - “Why did you pick this method over the alternative?”
 - “What would you change if the problem size doubled? Halved?”
 - “Explain a derivation/algorithm/result from Problem [Y] at the whiteboard.”

Part I — Large Computational Problems (pick 2 of 4; optional 3rd for bonus)

Pick any two of the four problems below for full credit (20 pts each = 40 pts). **Optional: a third problem for +10 pts bonus.**

If you submit work for all four problems, your best two count for the required 40 pts and the next-best counts as the +10 bonus. The fourth is ignored.

Problem 1.1. Markowitz portfolio optimization — a constrained QP. [20 pts]

A classic portfolio problem. You are given the (estimated) covariance matrix $\Sigma \in \mathbb{S}_{++}^n$ and expected returns $\mu \in \mathbb{R}^n$ for $n = 30$ assets in `data/portfolio30.npz` (Canvas). Solve the long-only, fully-invested mean-variance problem

$$\min_{\mathbf{w} \in \mathbb{R}^n} \frac{\gamma}{2} \mathbf{w}^\top \Sigma \mathbf{w} - \mu^\top \mathbf{w} \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{w} = 1, \mathbf{w} \geq \mathbf{0},$$

with risk-aversion parameter $\gamma = 5$.

Data. Load with:

```
import numpy as np
d = np.load('data/portfolio30.npz')
Sigma, mu = d['Sigma'], d['mu']
n = len(mu); gamma = 5.0
```

Tasks.

- CVXPY baseline.** Code the problem in CVXPY. Report optimum, runtime, and the solver chosen.
- Log-barrier interior-point method.** Implement an IPM from scratch by removing the inequality constraint $\mathbf{w} \geq \mathbf{0}$ and adding $-\frac{1}{t} \sum_i \log w_i$ to the objective. Solve the equality-constrained Newton subproblem at each t via the KKT system

$$\begin{bmatrix} \nabla_{\mathbf{w}}^2 L_t & A^\top \\ A & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{w} \\ \Delta \nu \end{bmatrix} = - \begin{bmatrix} \nabla_{\mathbf{w}} L_t \\ A\mathbf{w} - \mathbf{b} \end{bmatrix},$$

where $A = \mathbf{1}^\top$, $\mathbf{b} = 1$. Use the path $t \rightarrow t \cdot \mu$ with $\mu = 10$ until $n/t < 10^{-8}$. Report the iteration count of outer t -updates and inner Newton steps.

- Nesterov-accelerated projected gradient.** Use the projection onto the simplex

$$P_\Delta(\mathbf{v}) = \max(\mathbf{v} - \theta^*, \mathbf{0}), \quad \theta^* = \frac{1}{\rho} (\sum_{i \leq \rho} u_{(i)} - 1),$$

where $u_{(1)} \geq u_{(2)} \geq \dots \geq u_{(n)}$ are the sorted entries of $\mathbf{v}$ and $\rho = \max\{j : u_{(j)} - \frac{1}{j} (\sum_{i \leq j} u_{(i)} - 1) > 0\}$. (You will derive this formula in Problem 6; for now, use it as given.) Apply Nesterov-accelerated projected gradient. Report iterations to reach $f(\mathbf{w}_k) - f^* \leq 10^{-6}$.

- Algorithm-selection essay** (1–2 paragraphs). When does each method dominate? Consider three scenarios: (i) re-solving with new μ each minute (rebalancing in production), (ii) one-off allocation with high precision required, (iii) running 10,000 candidate Σ matrices in a sensitivity sweep.
- Visualize.** Plot the efficient frontier (expected return vs. standard deviation) by sweeping $\gamma \in \{0.5, 1, 2, 5, 10, 25, 100\}$. Annotate which method you used (and why) for each point.

What I'll ask on the oral: how does the IPM's central path differ from a homotopy on γ ? Why does the simplex projection have a closed form?

Solution:

Problem 1.2. Training a classifier — pick and defend an algorithm. [20 pts]

You are handed the UCI Wisconsin Diagnostic Breast Cancer dataset (`data/wdbc.npz`: $N = 569$ samples, $p = 30$ features, binary labels $y \in \{-1, +1\}^N$ for malignant (-1) vs. benign ($+1$); features are pre-standardized). Train an ℓ_2 -regularized logistic regression classifier

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \frac{1}{N} \sum_{i=1}^N \log(1 + \exp(-y_i(\mathbf{w}^\top \mathbf{a}_i + b))) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

with $\lambda = 0.01$.

Data. Load with:

```
import numpy as np
d = np.load('data/wdbc.npz')
A, y = d['A'], d['y']                                     # (569, 30) standardized ; (569,) in {-1,+1}
N, p = A.shape; lam = 0.01
# 80/20 split: use a fixed seed so your train/test results are reproducible
rng = np.random.default_rng(0)
perm = rng.permutation(N)
tr, te = perm[:int(0.8*N)], perm[int(0.8*N):]
A_tr, y_tr, A_te, y_te = A[tr], y[tr], A[te], y[te]
```

Tasks.

- Solve the training problem with *four* methods. Three are required and one is your choice.
 - CVXPY (formulate and solve).
 - L-BFGS via `scipy.optimize.minimize(method="L-BFGS-B")` with your analytically-derived gradient.
 - Plain SGD that you implement from scratch (pick your own step-size schedule and mini-batch size; document and justify your choices).
 - Your choice* of a fourth: Adam, Newton-CG, ADMM, Nesterov-accelerated gradient, or another method we covered.
- For each method: final training loss, final gradient norm, wall-clock time, and train/test accuracy on an 80/20 split. Put results in one table.
- Plot *training loss vs. wall-clock time* (not iterations!) for all four methods on the same axes. This is the plot that matters in practice.
- Algorithm-selection essay.** Suppose you have to retrain this model every morning on a growing dataset that reaches $N = 10^7, p = 10^5$ in a year. Which method from your four do you keep? What additional engineering (parallelism, sparsity, variance reduction, ...) becomes necessary as N and p grow? What about if the data is *distributed across multiple machines*? Write 2–3 paragraphs.
- Bonus** (+3 pts). Add ℓ_1 regularization (elastic net). Which of your four methods can still solve this directly, and which need modification?

What I'll ask on the oral: given your training plot, at what loss value is SGD the winner, and at what value does the second-order method overtake it? Why?

Solution:

Problem 1.3. Nonconvex optimization — IEEE 9-bus AC Optimal Power Flow. [20 pts]

What is AC OPF, and why does it matter? AC Optimal Power Flow is the dispatch problem solved every five minutes by every electric grid operator in the world (PJM, CAISO, MISO, ERCOT, NYISO, ...). The instance below is small, but it is structurally identical to the multi-trillion-dollar dispatch problem that keeps the lights on.

- A network of **buses** (substations) is connected by **transmission lines** of known impedance. The line topology and impedances combine into the **admittance matrix** $Y = G + jB$.
- Each bus has a **load** $(P_{d,i}, Q_{d,i})$ to be served (real and reactive demand).

- Each **generator** has capacity bounds and a quadratic cost-of-production curve $c_i(P_g) = c_{2,i}P_g^2 + c_{1,i}P_g + c_{0,i}$.
- The decision variables are voltage magnitudes $|V_i|$, voltage angles θ_i , and generator outputs $P_{g,i}, Q_{g,i}$ at every bus.
- The constraints are the AC power-balance equations (one per bus, real and reactive) plus voltage and generator-capacity bounds.

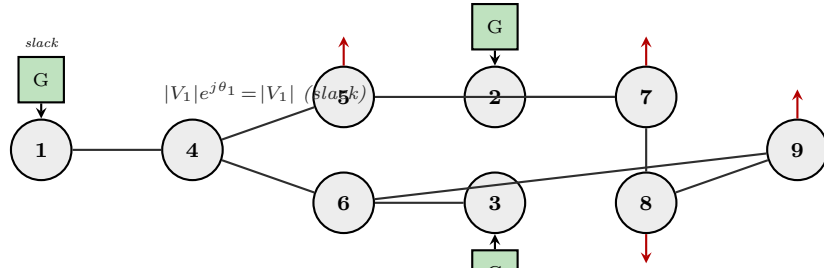
The variables, physically. AC voltage at bus i is a sinusoid $v_i(t) = |V_i| \cos(\omega t + \theta_i)$ — the phasor $V_i = |V_i|e^{j\theta_i}$ records its amplitude and phase. *Real power* P does mechanical work (lights, motors); *reactive power* Q supports voltage levels in inductive loads (transformers, motors). Both are conserved at every bus.

The slack-bus convention. Only *differences* of phase angles are physically observable, so we pin one bus's angle to $\theta_1 = 0$. (You couldn't measure absolute phase even if you wanted to.) Bus 1 is conventionally called the slack bus.

Why this is a hard problem. The power-balance equations contain products $|V_i||V_k| \cos(\theta_i - \theta_k)$, which are simultaneously bilinear (indefinite Hessian) and trigonometric (neither convex nor concave). The feasible set is a curved, nonconvex manifold, and the problem can have multiple local optima. For a 9-bus system this is benign; at 3000 buses (a small ISO footprint) it is hard.

Where it sits in the literature. The formulation traces to Carpentier (1962). Modern surveys: Cain–O'Neill–Castillo (2012), Frank–Steponavice–Rebennack (2012). Convex relaxations: Lavaei–Low (2012). Production solvers used inside ISOs: nonlinear IPMs (think IPOPT or commercial cousins) plus heuristic warm-starts.

Consider the IEEE 9-bus test system (the standard MATPOWER `case9` benchmark): 9 buses, 3 generators (at buses 1, 2, 3), 6 loads. Each bus $i \in \{1, \dots, 9\}$ has a complex voltage $V_i = |V_i|e^{j\theta_i}$. The per-unit admittance matrix $Y = G + jB$, generator limits, load demands, and quadratic cost coefficients are in `data/case9.npz`.



Bus i : complex voltage $V_i = |V_i|e^{j\theta_i}$. Lines carry power per the admittance matrix $Y = G + jB$.

Figure 1: Stylized topology of the IEEE 9-bus test system. Circles = buses; G boxes = generators (at buses 1, 2, 3); red arrows = loads (at buses 5, 7, 8, 9); gray lines = transmission branches. Bus 1 is the slack bus ($\theta_1 = 0$). The actual line impedances are stored in `data/case9.npz`.

Data. Load with:

```
import numpy as np
d = np.load('data/case9.npz')
G, B = d['G'], d['B'] # (9, 9) admittance (per-unit)
P_d, Q_d = d['P_d'], d['Q_d'] # (9,) bus loads (per-unit)
P_g_min, P_g_max = d['P_g_min'], d['P_g_max'] # generator real-power bounds
Q_g_min, Q_g_max = d['Q_g_min'], d['Q_g_max'] # reactive-power bounds
c2, c1, c0 = d['c2'], d['c1'], d['c0'] # quadratic gen cost coeffs
V_min, V_max = d['V_min'], d['V_max']
nb = G.shape[0] # 9
```

The AC OPF problem in polar form is:

$$\min_{|V|, \theta, P_g, Q_g} \sum_{i \in \mathcal{G}} c_i(P_{g,i})$$

subject to, for each bus i :

$$P_{g,i} - P_{d,i} = |V_i| \sum_k |V_k| (G_{ik} \cos(\theta_i - \theta_k) + B_{ik} \sin(\theta_i - \theta_k)),$$

$$Q_{g,i} - Q_{d,i} = |V_i| \sum_k |V_k| (G_{ik} \sin(\theta_i - \theta_k) - B_{ik} \cos(\theta_i - \theta_k)),$$

with bounds $|V|_i \in [V_i^{\min}, V_i^{\max}]$ ($\approx [0.9, 1.1]$ in the data), $\theta_1 = 0$ (slack), and $P_{g,i} \in [P_g^{\min}, P_g^{\max}]$. c_i is a quadratic generation cost.

The network data, bounds, loads, and cost coefficients are all in the data file.

Tasks.

- Formulate the problem in Pyomo** using `Vars` for voltages, angles, real powers, and reactive powers, and `Constraints` for the power balance at each bus. Explain briefly why this problem is **nonconvex** (identify the specific terms that fail convexity).
- Solve with IPOPT**. Start from the flat-voltage initial guess ($|V|_i = 1$, $\theta_i = 0$, generator at midpoint of its range). Report the optimal generation schedule, voltage profile, total cost, the IPOPT iteration count, and any exit messages.
- Sensitivity / multi-start**. Run IPOPT from 30 random starting points (voltages drawn uniformly in $[0.9, 1.1]$, angles in $[-\pi/6, \pi/6]$). Report the distribution of final objectives. Do you find a single solution, or multiple? If multiple, report the distinct cost values and describe how the voltage/angle configurations differ.
- Stress test**. Scale the loads P_d, Q_d by a factor $\alpha \in \{0.5, 1.0, 1.5, 2.0, 2.5\}$. For which α does IPOPT still return “Optimal Solution Found”? For which does it fail or return “Converged to locally infeasible”? Interpret what’s happening physically as α grows.
- Convex relaxation (optional, +5 pts)**. The *DC approximation* of AC OPF drops voltage magnitudes ($|V|_i \equiv 1$) and linearizes the power flow. Code the DC OPF and solve it with CVXPY. How close is the DC solution to the AC solution for the baseline case? When does it break down?

What I’ll ask on the oral: sketch the nonconvexity on a board. Which constraint is the geometric source? If this were a 3000-bus system, what algorithmic change would you prioritize?

Solution:

Problem 1.4. *Saddle-point dynamics — GDA vs. Extragradient.* [20 pts]

Why saddle-point problems matter. Min-max optimization is everywhere in modern computational science, and almost every method you’ve seen secretly involves a saddle point.

- Lagrangian duality.** Every constrained optimization problem you’ve solved this semester — the IPM in P1, the AC OPF in P3, every KKT system — has a Lagrangian $L(x, \lambda)$ whose stationary point is a saddle: min over the primal x , max over the dual λ . Methods that update x and λ together (augmented Lagrangian, ADMM, primal-dual IPMs) are saddle-point methods in disguise.
- Adversarial machine learning.** Training a robust classifier minimizes $\max_{\|\delta\| \leq \epsilon} L(w; x + \delta, y)$ — a saddle problem where the adversary picks the worst perturbation and the trainer picks the best weights.
- GANs.** Generator vs. discriminator is a min-max game: $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$.
- Distributionally robust optimization, fairness ML, two-player zero-sum games.** All saddle-point.

Why a 2D toy. The simplest case — a 2×2 bilinear example — is the canonical pathology that GAN trainers (re)discovered the hard way: naïve gradient descent-ascent *rotates* around the saddle without converging. The lesson generalizes; the 2D version is just easy to visualize and analyze. Part (e) below scales the same problem to $n = 50$ to confirm the lesson holds away from the toy.

Consider the convex-concave saddle problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} \max_{\mathbf{y} \in \mathbb{R}^n} f(\mathbf{x}, \mathbf{y}) = \frac{\mu}{2} \|\mathbf{x}\|^2 + \mathbf{x}^\top B \mathbf{y} - \frac{\mu}{2} \|\mathbf{y}\|^2, \quad B \in \mathbb{R}^{n \times n} \text{ given.}$$

The unique saddle point is at $(\mathbf{x}^*, \mathbf{y}^*) = (\mathbf{0}, \mathbf{0})$. We start with $n = 2$ for parts (a)–(d) and scale to $n = 50$ in part (e).

Two algorithms.

- **Gradient descent-ascent (GDA).** At each iterate $(\mathbf{x}_k, \mathbf{y}_k)$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla_{\mathbf{x}} f(\mathbf{x}_k, \mathbf{y}_k), \quad \mathbf{y}_{k+1} = \mathbf{y}_k + \eta \nabla_{\mathbf{y}} f(\mathbf{x}_k, \mathbf{y}_k).$$

- **Extragradient (Korpelevich, 1976).** Predictor + corrector:

$$\text{Predictor: } \tilde{\mathbf{x}}_k = \mathbf{x}_k - \eta \nabla_{\mathbf{x}} f(\mathbf{x}_k, \mathbf{y}_k), \quad \tilde{\mathbf{y}}_k = \mathbf{y}_k + \eta \nabla_{\mathbf{y}} f(\mathbf{x}_k, \mathbf{y}_k).$$

$$\text{Corrector: } \mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla_{\mathbf{x}} f(\tilde{\mathbf{x}}_k, \tilde{\mathbf{y}}_k), \quad \mathbf{y}_{k+1} = \mathbf{y}_k + \eta \nabla_{\mathbf{y}} f(\tilde{\mathbf{x}}_k, \tilde{\mathbf{y}}_k).$$

The key is that the *corrector* uses the gradient evaluated at the lookahead $(\tilde{\mathbf{x}}_k, \tilde{\mathbf{y}}_k)$, not at $(\mathbf{x}_k, \mathbf{y}_k)$.

Tasks. Use $B = \begin{pmatrix} 1.0 & 0.5 \\ -0.3 & 1.0 \end{pmatrix}$, $\mathbf{x}_0 = (1.0, 0.5)^\top$, $\mathbf{y}_0 = (0.5, -0.3)^\top$, $\eta = 0.1$, 200 iterations.

- GDA on three regimes.** Implement GDA. Run it for $\mu \in \{1.0, 0.1, 0.0\}$. For each, plot the iterate trajectory in the 2D plane (you may project onto the (x_1, y_1) coordinates). Verify that GDA converges for $\mu > 0$ and *rotates without converging* for $\mu = 0$.
- Extragradient on the same three regimes.** Implement extragradient. Show that it converges in *all three* cases including $\mu = 0$. Plot the trajectories alongside the GDA trajectories.
- Step-size sweep at $\mu = 0$ (purely bilinear).** Vary $\eta \in \{0.01, 0.1, 0.5, 1.0\}$. For GDA, what is the largest η that even keeps the iterates bounded over 200 iterations? For extragradient, plot the convergence rate (distance to saddle vs. iteration, log scale) for each η . Which η wins?
- Theory (1 paragraph each).**
 - For $\mu = 0$, write the linearized GDA dynamics $z_{k+1} = M z_k$ where $z = (\mathbf{x}; \mathbf{y})$. Compute the eigenvalues of M . Explain why GDA fails (hint: $|\lambda(M)| > 1$).
 - Why does extragradient succeed? Hint: write the linearized extragradient dynamics in the form $z_{k+1} = M_{EG} z_k$ and show that $|\lambda(M_{EG})| < 1$ for sufficiently small η .
- Scale up to $n = 50$.** Generate a random $B \in \mathbb{R}^{50 \times 50}$ (entries i.i.d. standard normal, divided by $\sqrt{n}$ so $\|B\|_2 \approx 1$), random initial $\mathbf{x}_0, \mathbf{y}_0 \in \mathbb{R}^{50}$, $\mu = 0$, $\eta = 0.1$. Run 500 iterations of GDA and 500 of extragradient. Plot $\|(\mathbf{x}_k, \mathbf{y}_k)\|$ versus k on a log scale, both methods on the same axes. Confirm: GDA still fails to converge (the iterate norm grows or oscillates), extragradient still drives $\|z_k\|$ toward zero. The qualitative dynamics from the 2D case carry over.

What I'll ask on the oral: sketch the GDA iterate trajectory on $f(x, y) = xy$ from $(1, 0)$ on a whiteboard. What is the period of the rotation as a function of η ?

Solution:

Part II — Theory Problems (short)

Problem 1.5. *Duality and KKT for a structured NLP.* [20 pts]

Consider

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \mathbf{x}^\top \mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0,$$

where $A \in \mathbb{R}^{m \times n}$ has full row rank ($m < n$) and there exists $\bar{\mathbf{x}} > 0$ with $A\bar{\mathbf{x}} = \mathbf{b}$ (strict feasibility).

- [5 pts] Write the Lagrangian. Derive the KKT conditions.
- [5 pts] Derive the dual problem explicitly. Is strong duality guaranteed, and why?
- [5 pts] Suppose we add a *redundant* equality $\mathbf{b}^\top \mathbf{y} - \mathbf{b}^\top \mathbf{y} = 0$ as a new row of A (so a copy-row of A duplicated with its own right-hand side). What goes wrong with the KKT matrix? How does IPOPT's *inertia correction* address it?
- [5 pts] Using the log barrier $-\sum_i \log x_i$, write the perturbed KKT system and show that the complementarity $x_i s_i = \mu$ for every i *plus* the equality $A\mathbf{x} = \mathbf{b}$ determines a unique point for each $\mu > 0$ (the central path).

Solution:

Problem 1.6. *The simplex projection from KKT — proof and complexity.* [20 pts]

Problem 1 used the closed-form simplex projection

$$P_\Delta(\mathbf{v}) = \arg \min_{\mathbf{w} \in \Delta} \frac{1}{2} \|\mathbf{w} - \mathbf{v}\|^2, \quad \Delta = \{\mathbf{w} : \mathbf{w} \geq \mathbf{0}, \mathbf{1}^\top \mathbf{w} = 1\}.$$

Here you will prove that the formula given in Problem 1 is correct, then analyze its computational cost.

- [5 pts] Write the Lagrangian for the projection QP. Identify the dual variable $\theta \in \mathbb{R}$ for the equality constraint $\mathbf{1}^\top \mathbf{w} = 1$ and the dual variables $\mathbf{s} \in \mathbb{R}_{\geq 0}^n$ for the inequality constraints $\mathbf{w} \geq \mathbf{0}$.
- [5 pts] Write the KKT conditions. Use stationarity together with primal/dual feasibility and complementary slackness to show that

$$w_i^* = \max(v_i - \theta^*, 0)$$

for some scalar θ^* that depends on $\mathbf{v}$.

- [5 pts] Show that θ^* is the unique solution of the scalar equation

$$g(\theta) := \sum_{i=1}^n \max(v_i - \theta, 0) = 1.$$

Argue uniqueness via monotonicity. Specify the interval on the real line in which θ^* must lie.

- [3 pts] Sort $\mathbf{v}$ in descending order so $u_{(1)} \geq u_{(2)} \geq \dots \geq u_{(n)}$. Let $\rho = |\{i : w_i^* > 0\}|$ be the size of the active set. Show that

$$\theta^* = \frac{1}{\rho} \left(\sum_{i=1}^{\rho} u_{(i)} - 1 \right), \quad \rho = \max \left\{ j \in \{1, \dots, n\} : u_{(j)} - \frac{1}{j} \left(\sum_{i \leq j} u_{(i)} - 1 \right) > 0 \right\}.$$

- [2 pts] **Algorithmic complexity.** State the time and space complexity of the algorithm in part (d) (the sort-based version). Is $O(n)$ time achievable, and how (sketch the idea, no proof needed)? Compare both to solving the projection as a generic QP via an interior-point method — give an asymptotic complexity for the IPM.

Solution:

Oral Defense — Format and Rubric [20 pts]

You will sign up for an **8-minute slot** during the **Monday, May 11, 2:05–4:05 PM** window. Come prepared with:

- A laptop (or printed copy) with your report and code.
- Paper and pencil — some questions will ask you to sketch or derive.

Format. I will pick *one* of your computational problems and ask four short questions about it. The four questions correspond directly to the four rubric rows below, so the structure is predictable.

Question types

1. **Algorithm-choice question.** “You chose Nesterov-accelerated PG over the IPM for Problem 1. Walk me through why, and what would change your mind.”
2. **Derivation at the board.** “On the board: derive the gradient you used in Problem 2,” or “write down the KKT system for Problem 3.”
3. **“What if” question.** “Suppose the problem in Problem 3 had 3000 buses instead of 9. What breaks, and what do you do?”
4. **Synthesis question.** “Of all the methods we covered this semester, which one surprised you most? Which one would you expect to reach for first in the real world? Why?”

Rubric (20 pts)

Each of the four questions is worth **5 pts**, scored on the descriptive bands below.

	Excellent (5)	Solid (4)	Adequate (3)	Limited (≤ 2)
1. Algorithm choice	Articulates the trade-off cleanly; names the alternative and the conditions under which it would win; cites a number from their report to back the claim.	States a correct reason; can name the alternative; reasoning is right but high-level.	Defends the choice with a textbook reason but doesn't engage the trade-off; vague about alternatives.	Cannot defend the choice independently of the report or hedges with "the AI suggested it."
2. Derivation	Reproduces the gradient / KKT / algorithm step at the board fluently, with correct notation, in < 2 minutes.	Derives correctly with one minor slip and self-corrects when prompted.	Gets the structure right but leaves gaps or signs wrong; needs a hint to finish.	Cannot reconstruct a derivation that appears in their own report.
3. "What if" / scaling	Identifies the bottleneck (per-iteration cost, factorization, communication, etc.) and proposes a concrete alternative with rationale; engages the regime change.	Identifies the bottleneck correctly; proposed alternative is reasonable but high-level.	Knows something will break but vague on which step or why; suggests a generic remedy.	Cannot reason past the problem instance they actually ran.
4. Synthesis	Connects across problems / theory / outside courses; offers an opinion grounded in the semester's evidence; raises an interesting question of their own.	Makes a clear connection between two parts of the course; opinion is reasoned.	Names a method but mostly recites; thin on "why."	Cannot abstract beyond the immediate problem.

Scoring philosophy. There are no trick questions. I'm trying to find out what you *understand*, not what you can recite. If you don't know something, say so and reason out loud — partial credit is generous when you show your thinking. Scores typically cluster in the 14–19 range; a 20 means I learned something from talking to you, and anything below 10 indicates you couldn't defend the work as your own.

AI-disclosure cross-check. If your written report disclosed AI use on a particular problem, I may ask you to walk me through the part you used AI on, in your own words. Inability to do so is a flag — it does not by itself fail you, but it shifts the rest of the rubric toward the lower end.